

EVENT DRIVEN NOTIFICATION ENGINE WITH MULTI-CHANNEL DELIVERY

Category: Financial Technology | Backend Engineering | Event-Driven Systems | Notification Infrastructure

Timeline: 15 Days | AI-Accelerated Development

Target: Undergraduate / Engineering (CS/IT) / MBA (Tech/Analytics) / Backend Developer Interns

TASK:

Design and implement an event-driven notification back-end that processes 25+ financial event types (transaction alerts, margin calls, SIP reminders, price alerts, regulatory notices) and delivers them across SMS, email, push notification, WhatsApp, and in-app channels. Build user preference management, TRAI DND compliance, delivery tracking, dead-letter queue with retry strategy, template engine with personalisation and localisation, frequency capping, quiet hours enforcement, and a real-time analytics pipeline. Transfer GitHub repository to @ZethetaIntern on Day 15.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

PART E: NO-CODE TOOLS & PLATFORMS GUIDE

PART F: SUBMISSION PROTOCOL

Important Note:

This Project is designed, developed, and administered solely by Zetheta Algorithms Private Limited ("Zetheta") for the purpose of assessing candidates, students, or event participants on behalf of engaging employers, institutes, or event organisers. Completion of this Project does not guarantee employment, academic outcomes, awards, or any other result - all such decisions remain at the sole discretion of the respective engaging party. Zetheta may use AI-assisted tools to evaluate submissions; such outputs are indicative in nature and may be inaccurate or incomplete. By participating, you consent to your performance data, scores, and submissions being shared with relevant engaging parties for assessment and decision-making purposes.

EXECUTIVE SUMMARY

Event-Driven Notification Engine is an enterprise-scale backend engineering project focused on designing and implementing a high-performance, multi-channel notification system for financial services. Participants build a scalable architecture capable of processing 25+ financial event types-including transaction alerts, margin calls, SIP reminders, price alerts, and regulatory notifications-and delivering them across SMS, email, push notifications, WhatsApp, and in-app channels.

Working with a real-world fintech context, the project simulates production-grade systems used by brokerage platforms and wealth management applications. It introduces participants to event-driven architecture (EDA), message brokers like Kafka and RabbitMQ, and distributed system design patterns such as event sourcing, CQRS, saga orchestration, and dead-letter queues.

The project transforms raw event streams into intelligent, user-personalised communication through advanced backend capabilities including channel routing, user preference management, TRAI DND compliance, frequency capping, quiet hours enforcement, and real-time delivery tracking. Participants also build a template engine supporting personalisation and localisation across multiple languages, ensuring relevant and timely communication for diverse user segments.

The core challenge participants must solve is: how to design a fault-tolerant, scalable notification system that ensures high delivery reliability, regulatory compliance, and optimal user engagement under heavy load conditions. This involves handling millions of notifications daily, managing provider failures, preventing notification fatigue, and ensuring critical alerts are delivered within strict latency constraints.

By the end of the project, participants will have built a production-ready backend system capable of handling real-time financial events at scale, while balancing performance, cost optimisation, compliance, and user experience-skills directly applicable to modern fintech and distributed systems engineering roles.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

SECTION A1: EVENT DRIVEN ARCHITECTURE FUNDAMENTALS

A1.1 What is Event-Driven Architecture (EDA)?

Event-Driven Architecture is a software design paradigm in which the flow of the programme is determined by events - significant changes in state. In the context of a notification engine, an event could be a stock price crossing a threshold, a transaction being executed, or a margin call being triggered. Unlike traditional request-response patterns, EDA decouples event producers from event consumers, enabling asynchronous processing, horizontal scalability, and fault tolerance.

In a financial services context, EDA is critical because financial events are inherently asynchronous and time-sensitive. A margin call cannot wait for a synchronous API response - it must be processed immediately and delivered through the fastest available channel. Similarly, a price alert for 100,000 users watching the same stock cannot be processed sequentially; the system must fan out notifications in parallel while maintaining delivery guarantees.

A1.2 Core EDA Patterns for Notification Systems

A. Event Sourcing

Every state change is captured as an immutable event in an append-only log. For a notification engine, this means every notification request, delivery attempt, delivery confirmation, and failure is recorded. This provides a complete audit trail — essential for regulatory compliance under SEBI and TRAI regulations. Event sourcing also enables replay: if a channel provider was down and notifications were queued, the system can replay events to reconcile state after recovery.

B. CQRS (Command Query Responsibility Segregation)

Separate the write model (ingesting events, processing notifications) from the read model (querying delivery status, generating analytics). The write side processes high-throughput event ingestion into Kafka/RabbitMQ, while the read side serves dashboards and analytics from denormalised views in Redis or Elasticsearch. This separation allows each side to scale independently — the write side during market hours when event volume spikes, the read side when compliance teams run morning reports.

C. Saga Pattern

Multi-channel delivery is a distributed transaction. Sending a margin call via SMS, email, and push simultaneously requires coordination. If the SMS fails, should the system retry immediately or fall back to WhatsApp? The Saga pattern manages this by defining a sequence of local transactions with compensating actions. For a critical notification like a margin call, the saga might define: attempt push notification → if undelivered after 30 seconds, attempt SMS → if SMS fails, attempt WhatsApp → if all fail, escalate to manual call queue.

D. Dead Letter Queue (DLQ)

Messages that cannot be processed after a configurable number of retries are moved to a Dead Letter Queue. In a notification engine, DLQ entries represent critical failures: an SMS that could not be delivered after 5 retries, a push notification to an expired device token, or an email bounced permanently. The DLQ must be monitored with alerts and processed by an operations team or automated remediation workflows.

A1.3 Message Brokers: Kafka vs RabbitMQ vs Redis Streams

Choosing the right message broker is one of the most consequential architectural decisions for a notification engine. Each has distinct characteristics:

Apache Kafka	RabbitMQ	Redis Streams
Distributed log-based; messages persisted to disk; consumer groups for parallel processing	AMQP-based; sophisticated routing with exchanges and bindings; message acknowledgement	In-memory stream with consumer groups; lightweight; sub-millisecond latency
Throughput: 1M+ messages/sec per partition	Throughput: 50K-100K messages/sec	Throughput: 500K+ messages/sec
Retention: Configurable (hours to infinite)	Retention: Until acknowledged	Retention: Configurable with MAXLEN
Best for: High-volume event streaming, replay capability, audit logs	Best for: Complex routing, priority queues, request-reply patterns	Best for: Real-time low-latency processing, caching layer integration
Use here: Primary event bus for ingestion	Use here: Delivery routing, priority queues, DLQ	Use here: Rate limiting, caching, real-time counters

SECTION A2: FINANCIAL EVENT TAXONOMY

A2.1 The 25+ Financial Event Types

A well-designed notification engine begins with a comprehensive event taxonomy. Financial events vary dramatically in urgency, regulatory requirements, and user expectations. The following taxonomy classifies events across five dimensions: urgency, regulatory mandate, personalisation depth, channel preference, and frequency characteristics.

A2.2 Category 1: Transaction Events (TXNX-*)

Event Code	Event Name	Urgency	Regulatory	Channels	Max Latency	Personalisation
TXNX-001	Buy Order Executed	HIGH	SEBI Mandatory	SMS, Push, Email	< 30 seconds	Stock name, qty, price, total, portfolio impact
TXNX-002	Sell Order Executed	HIGH	SEBI Mandatory	SMS, Push, Email	< 30 seconds	Stock name, qty, price, P&L, tax implication
TXNX-003	Order Rejected	HIGH	Required	Push, SMS	< 15 seconds	Reason code, alternative action, support link
TXNX-004	Dividend Credited	MEDIUM	Informational	Email, Push	< 1 hour	Company, amount, record date, bank account
TXNX-005	Funds Deposited	HIGH	Banking Reg	SMS, Push	< 60 seconds	Amount, source, available balance

ADD EXPERIENCE TO KNOWLEDGE

A2.3 Category 2: Risk & Margin Events (RISK-*)

Event Code	Event Name	Urgency	Regulatory	Channels	Max Latency	Personalisation
RISK-001	Margin Call Warning	CRITICAL	SEBI Mandatory	SMS, Push, Call	< 10 seconds	Shortfall amount, deadline, liquidation risk
RISK-002	Margin Shortfall	CRITICAL	SEBI Mandatory	SMS, Push, Email, Call	< 5 seconds	Shortfall, action required, auto-square-off time
RISK-003	Position Squared Off	CRITICAL	SEBI Mandatory	SMS, Push, Email	< 15 seconds	Positions closed, P&L impact, remaining positions
RISK-004	Portfolio Risk Alert	HIGH	Advisory	Push, Email	< 5 minutes	Risk metric breached, affected holdings, suggestion
RISK-005	Concentration Alert	MEDIUM	Advisory	Email, In-App	< 1 hour	Over-concentrated sector/stock, % allocation

A2.4 Category 3: SIP & Investment Events (SIPX-*)

Event Code	Event Name	Urgency	Regulatory	Channels	Max Latency	Personalisation
SIPX-001	SIP Due Reminder	MEDIUM	Best Practice	Push, WhatsApp	T-3 days	Fund name, amount, date, bank balance check
SIPX-002	SIP Executed	HIGH	AMFI Required	SMS, Email	< 2 hours	Fund, units allotted, NAV, total investment
SIPX-003	SIP Failed	HIGH	Required	SMS, Push, Email	< 30 minutes	Reason, retry date, action required

SIPX-004	SIP Step-Up Reminder	LOW	Advisory	Email, In-App	Annual	Current amount, suggested increase, goal impact
SIPX-005	Goal Milestone Reached	LOW	Engagement	Push, In-App	< 4 hours	Goal name, % complete, projected completion

A2.5 Category 4: Market & Price Events (MKTX-*)

Event Code	Event Name	Urgency	Regulatory	Channels	Max Latency	Personalisation
MKTX-001	Price Alert Triggered	HIGH	User-Set	Push, SMS	< 15 seconds	Stock, target price, current price, direction
MKTX-002	Circuit Breaker Hit	CRITICAL	Exchange Mandate	Push, SMS	< 10 seconds	Stock, circuit level, trading halt duration
MKTX-003	Market Open/Close	LOW	Engagement	Push	At event time	Index levels, portfolio overnight change
MKTX-004	52-Week High/Low	MEDIUM	Advisory	Push, Email	< 30 minutes	Stock, new milestone, holding status
MKTX-005	Earnings Announcement	MEDIUM	Advisory	Email, In-App	T-1 day	Company, date, expected EPS, historical

ADD EXPERIENCE TO KNOWLEDGE

A2.6 Category 5: Regulatory & Compliance Events (REGX-*)

Event Code	Event Name	Urgency	Regulatory	Channels	Max Latency	Personalisation
REGX-001	KYC Expiry Warning	HIGH	SEBI Mandatory	SMS, Email, Push	T-30, T-15, T-7 days	Expiry date, documents needed, submission link
REGX-002	Nominee Update Reminder	MEDIUM	SEBI Circular	Email, Push	Quarterly	Current nominee status, deadline, link

REGX-003	Contract Note Generated	HIGH	SEBI Mandatory	Email	T+1 EOD	Trade date, PDF attachment, summary
REGX-004	Tax Statement Available	MEDIUM	IT Act Required	Email, In-App	Quarterly	Period, download link, key figures
REGX-005	Regulatory Policy Change	LOW	Informational	Email	< 24 hours	Change summary, impact on user, effective date



ZeTheta

ADD EXPERIENCE TO KNOWLEDGE

SECTION A3: MULTI CHANNEL DELIVERY ARCHITECTURE

A3.1 Channel Characteristics and Selection Matrix

Each delivery channel has distinct characteristics that affect routing decisions. Understanding these is critical for building an intelligent routing engine:

Characteristic	SMS	Email	Push	WhatsApp	In-App	IVR/Call	Webhook
Avg Delivery Time	3-10 sec	1-60 sec	< 1 sec	2-5 sec	Real-time	15-45 sec	< 500ms
Delivery Rate	95-98 %	85-95%	60-80%	90-95%	100%*	70-85%	99%+
Cost per Message	₹0.15-0.25	₹0.01-0.05	Free**	₹0.30-0.80	Free	₹1.50-3.00	Free
Regulatory Constraint	TRAI DND	CAN-S PAM	OS Perms	WhatsApp TOS	None	TRAI DND	None
Rich Content	160 chars	Full HTML	Title+Body	Media+Buttons	Full UI	Voice Only	JSON
User Must Be Online	No	No	No***	No	Yes	Must Answer	Server-side

* When the user opens the app. ** FCM/APNs have generous free tiers. *** Stored by FCM/APNs for limited time.

A3.2 Channel Routing Decision Engine

The routing engine must make channel selection decisions based on multiple factors evaluated in a priority-weighted scoring model:

Priority 1: Regulatory Mandate (Non-Negotiable)

If an event has a regulatory channel requirement (e.g., SEBI mandates SMS for margin calls), that channel **MUST** be used regardless of user preferences. The routing engine must have a regulatory override layer that supersedes all other routing logic.

Priority 2: User Preferences (Configurable)

Users can specify channel preferences per event category. A user might want transaction alerts via push but market updates via email digest. The preference system must support granular per-category, per-channel configurations with sensible defaults.

Priority 3: Delivery Optimisation (Intelligent)

Based on historical delivery data, the engine should prefer channels with higher delivery rates for each user. If a user consistently ignores push notifications but opens WhatsApp messages, the engine should route non-regulatory notifications to WhatsApp. This requires maintaining per-user, per-channel engagement metrics.

Priority 4: Cost Optimisation (Business Logic)

When multiple channels have similar delivery characteristics, prefer the lower-cost option. Route non-urgent notifications to email or in-app rather than SMS. Implement cost budgets per user per day/month to prevent runaway notification costs.

A3.3 Delivery Provider Integration Patterns

Your system must abstract delivery providers behind a unified interface. Each provider implementation must handle:

- Authentication (API keys, OAuth tokens, webhook signatures)
- Rate limiting (provider-specific throttling with backoff)
- Response parsing (delivery receipts, failure codes, status callbacks)
- Failover (primary → secondary provider per channel)
- Circuit breaking (disable a provider if failure rate exceeds threshold)

```
interface DeliveryProvider {  
    send(notification: PreparedNotification):  
    Promise<DeliveryResult>;  
    getStatus(externalId: string): Promise<DeliveryStatus>;  
    validateRecipient(address: string):  
    Promise<ValidationResult>;  
    getQuota(): Promise<QuotaInfo>;  
}
```


SECTION A4: TEMPLATE ENGINE & PERSONALISATION

A4.1 Template Architecture

The template engine is the bridge between raw event data and user-facing messages. It must support:

- Handlebars/Mustache-style variable interpolation for dynamic content injection
- Conditional rendering based on event attributes (e.g., show P&L only if user has the holding)
- Localisation with i18n support for at minimum English, Hindi, Marathi, Tamil, and Telugu
- Channel-specific formatting (160-char SMS vs. rich HTML email vs. JSON push payload)
- Template versioning with A/B testing capability
- Fallback chains: if a localised template is missing, fall back to English

A4.2 Template Schema Design

Each template must be defined as a configuration object, not hardcoded:

```
{
  "template_id": "TXNX-001-v2",
  "event_type": "TXNX-001",
  "version": 2,
  "channels": {
    "sms": {
      "body": "{{stock_name}} BUY executed: {{qty}} shares @ {{price}}. Total: {{total}}. Portfolio: {{portfolio_value}}. -{{app_name}}",
      "max_length": 160,
      "sender_id": "ZTHETA"
    },
    "email": {
      "subject": "Order Executed: {{qty}} {{stock_name}} @ {{price}}",
      "html_template": "templates/txnx-001/email.hbs",
      "text_fallback": "templates/txnx-001/email.txt"
    },
    "push": {
```

```
"title": "Buy Order Executed",
"body": "{{qty}} {{stock_name}} @ ₹{{price}}",
"data": { "action": "open_portfolio", "order_id":
"{{order_id}}" },
},
"whatsapp": {
  "template_name": "order_executed_v2",
  "language": "{{user_language}}",
  "components": [{ "type": "body", "parameters":
["{{stock_name}}", "{{qty}}", "{{price}}"] }]
}
},
"personalisation_fields": ["stock_name", "qty", "price",
"total", "portfolio_value", "app_name", "order_id",
"user_language"],
"localisation": {
  "hi": { "sms": { "body": "{{stock_name}} खरीद आदेश पूरा:
{{qty}} शेयर @ {{price}}..." } }
}
}
```

A4.3 Personalisation Pipeline

Personalisation goes beyond simple variable substitution. The engine must:

- Resolve user context: name, language preference, timezone, account type, risk profile
- Compute derived fields: P&L calculation, portfolio impact %, tax implications
- Apply formatting rules: currency formatting per locale (₹1,00,000 vs \$100,000), date formatting
- Truncate gracefully: SMS body must fit 160 chars; if personalised content exceeds, truncate intelligently with ellipsis and a link
- Handle missing data: if a personalisation field is null, use a sensible default or omit the section

SECTION A5: USER PREFERENCE MANAGEMENT

A5.1 Preference Hierarchy

User preferences operate in a layered hierarchy with inheritance and override capabilities:

- Layer 1 (System Defaults): Applied to all users. Defines the baseline notification behaviour for each event type. Example: all CRITICAL events use SMS + Push.
- Layer 2 (Segment Overrides): Applied based on user segments (e.g., premium users, active traders, passive investors). Premium users might get WhatsApp for all events.
- Layer 3 (User Explicit Preferences): User-configured preferences from the settings screen. A user can disable email for market updates but keep it for transaction confirmations.
- Layer 4 (Regulatory Override): Non-negotiable requirements that override all user preferences. SEBI-mandated notifications CANNOT be disabled by the user.

A5.2 Preference Data Model

```
CREATE TABLE user_preferences (  
    user_id UUID NOT NULL REFERENCES users(id),  
    event_category VARCHAR(10) NOT NULL, -- e.g., 'TXNX',  
    'RISK', 'SIPX'  
    event_type VARCHAR(10), -- NULL means all events in category  
    channel VARCHAR(20) NOT NULL, -- 'sms', 'email', 'push',  
    'whatsapp', 'in_app'  
    enabled BOOLEAN NOT NULL DEFAULT TRUE,  
    quiet_hours_override BOOLEAN DEFAULT FALSE, -- can this  
    bypass quiet hours?  
    digest_mode VARCHAR(20) DEFAULT 'immediate', -- 'immediate',  
    'hourly', 'daily'  
    priority_override INTEGER, -- user can upgrade/downgrade  
    priority  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    PRIMARY KEY (user_id, event_category, COALESCE(event_type,  
    '*'), channel)  
);
```

SECTION A6: TRAI DND COMPLIANCE & FREQUENCY CAPPING

A6.1 TRAI DND Registry Compliance

India's Telecom Regulatory Authority (TRAI) mandates compliance with the National Do Not Call (DND) registry for all commercial communications sent via SMS or voice. Your system **MUST** implement:

- **DND Registry Lookup:** Before sending ANY SMS, check the user's DND status against the scrubbing database. Maintain a local cache refreshed daily from the operator-provided DND list.
- **DND Category Classification:** TRAI defines seven DND preference categories (Banking/Insurance/Financial Products, Real Estate, Education, Health, Consumer Goods, Communication/Broadcasting, Tourism). Financial notifications fall under Category 1 (Banking/Insurance/Financial Products). Users who have opted into Category 1 can receive transactional AND promotional financial messages.
- **Transactional vs Promotional:** Transactional messages (order confirmations, OTPs, margin calls) are generally exempt from DND. However, promotional messages (SIP step-up suggestions, new fund recommendations) are NOT exempt. Your system must classify each notification as TRANSACTIONAL or PROMOTIONAL and apply DND filtering accordingly.
- **Consent Management:** Maintain explicit opt-in records with timestamps. For WhatsApp, maintain separate opt-in as per WhatsApp Business Policy. For email, comply with the Information Technology Act 2000 and its amendments regarding electronic consent.

A6.2 Frequency Capping Engine

Frequency capping prevents notification fatigue - the primary driver of the 23% opt-out rate in the simulation scenario. Implement multi-dimensional capping:

Cap Dimension	Limit	Window	Override Condition
Global per-user daily	Max 12 notifications/day	Rolling 24 hours	CRITICAL events bypass
Per-channel daily	SMS: 5, Push: 8, Email: 3	Rolling 24 hours	Regulatory mandates bypass
Per-category hourly	Max 3 per category/hour	Rolling 60 minutes	Margin calls always sent
Cooldown between same-type	Min 15 minutes gap	Per event type	Price alerts configurable

Weekly digest threshold	If > 8/day, suggest digest	Rolling 7 days	User can override
--------------------------------	----------------------------	----------------	-------------------

A6.3 Quiet Hours Enforcement

Quiet hours prevent notifications during user-specified rest periods. Default quiet hours are 21:00 – 08:00 in the user's timezone. Implementation requirements:

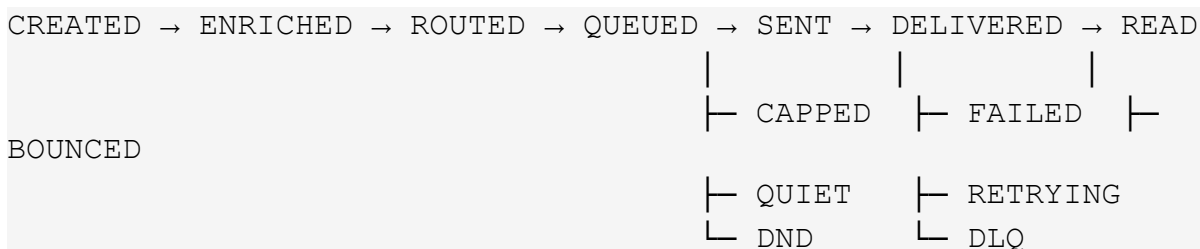
- Per-user timezone resolution (store as IANA timezone, e.g., 'Asia/Kolkata')
- Customisable quiet hours per user with a minimum 6-hour enforced quiet window
- Queue notifications during quiet hours and deliver at the start of the active window
- CRITICAL events (margin calls, security alerts) bypass quiet hours with an audit log entry
- Aggregate queued notifications into a morning digest if count exceeds 5



SECTION A7: DELIVERY TRACKING & ANALYTICS PIPELINE

A7.1 Notification Lifecycle State Machine

Every notification transitions through a defined state machine. Your implementation must track and persist every state transition:



Each state transition must be recorded with a timestamp, the actor (system component), and any relevant metadata (e.g., failure reason, retry count, provider response).

A7.2 Analytics Metrics

The analytics pipeline must compute and expose the following metrics in real-time (< 30 second lag) and historical (batch) modes:

- **Delivery Rate:** Percentage of notifications successfully delivered per channel, event type, and time period
- **Open/Read Rate:** Percentage of delivered notifications that were opened (email) or read (push, in-app)
- **Click-Through Rate:** For notifications with action buttons/links, the percentage that resulted in a user action
- **Time-to-Delivery:** P50, P95, P99 latency from event occurrence to delivery confirmation per channel
- **Failure Rate:** Percentage of notifications that ended in FAILED or DLQ state, broken down by failure reason
- **Opt-Out Rate:** Rolling 30-day opt-out rate per channel and event category (target: reduce from 23% to < 8%)
- **Cost per Notification:** Average cost per delivered notification per channel
- **Frequency Cap Hit Rate:** How often notifications are being capped (indicates potential over-notification)

SECTION A8: DATABASE SCHEMA DESIGN

A8.1 Core Tables

Your PostgreSQL schema must support the full notification lifecycle with proper indexing for query performance:

```
-- Core notification table (partitioned by created_at for
time-series queries)
CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    event_type VARCHAR(10) NOT NULL,
    event_id VARCHAR(50) NOT NULL, -- external event reference
    user_id UUID NOT NULL REFERENCES users(id),
    channel VARCHAR(20) NOT NULL,
    priority INTEGER NOT NULL DEFAULT 5, -- 1=CRITICAL, 5=LOW
    status VARCHAR(20) NOT NULL DEFAULT 'CREATED',
    template_id VARCHAR(50) NOT NULL,
    template_version INTEGER NOT NULL,
    personalisation_data JSONB NOT NULL,
    rendered_content JSONB, -- after template rendering
    provider VARCHAR(30), -- delivery provider used
    external_id VARCHAR(100), -- provider's message ID
    delivery_attempts INTEGER DEFAULT 0,
    max_retries INTEGER DEFAULT 3,
    next_retry_at TIMESTAMPTZ,
    delivered_at TIMESTAMPTZ,
    read_at TIMESTAMPTZ,
    failed_reason TEXT,
    cost_paisa INTEGER, -- cost in paisa for billing
    metadata JSONB, -- extensible metadata
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (created_at);

-- State transition audit log
CREATE TABLE notification_state_log (
    id BIGSERIAL PRIMARY KEY,
    notification_id UUID NOT NULL,
```

```
from_status VARCHAR(20),
to_status VARCHAR(20) NOT NULL,
actor VARCHAR(50) NOT NULL, -- e.g., 'routing_engine',
'sms_provider', 'retry_worker'
metadata JSONB,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_nsl_notification ON
notification_state_log(notification_id);
CREATE INDEX idx_nsl_created ON
notification_state_log(created_at);

-- Dead Letter Queue
CREATE TABLE dead_letter_queue (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  notification_id UUID NOT NULL REFERENCES notifications(id),
  original_event JSONB NOT NULL,
  failure_reason TEXT NOT NULL,
  retry_count INTEGER NOT NULL,
  last_error TEXT,
  resolved BOOLEAN DEFAULT FALSE,
  resolved_by VARCHAR(50),
  resolved_at TIMESTAMPTZ,
  resolution_action VARCHAR(50), -- 'retried', 'discarded',
'manual_send'
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

A8.2 Indexing Strategy

Proper indexing is critical for a system processing 2M+ notifications daily. Your schema must include:

- Composite index on (user_id, status, channel) for user notification queries
- Partial index on (status) WHERE status IN ('QUEUED', 'RETRYING') for worker queries
- BRIN index on created_at for time-range scans on the partitioned table
- GIN index on personalisation_data for JSONB queries
- Index on (event_type, created_at) for analytics aggregations

SECTION A9: RETRY STRATEGY & DEAD LETTER QUEUE

A9.1 Exponential Backoff with Jitter

Failed deliveries must be retried with exponential backoff to avoid overwhelming providers during partial outages:

```
retryDelay = min(baseDelay * 2^attempt + randomJitter(0,
1000), maxDelay)
// baseDelay: 1000ms, maxDelay: 300000ms (5 minutes)
// Attempt 1: ~1-2 seconds
// Attempt 2: ~2-3 seconds
// Attempt 3: ~4-5 seconds
// Attempt 4: ~8-9 seconds
// Attempt 5: ~16-17 seconds (then move to DLQ)
```

Different event priorities have different retry configurations:

Priority	Max Retries	Base Delay	Max Delay
CRITICAL (1)	10	500ms	60 seconds
HIGH (2)	5	1 second	5 minutes
MEDIUM (3)	3	5 seconds	30 minutes
LOW (5)	2	30 seconds	2 hours

SECTION A10: SECURITY CONSIDERATIONS

A10.1 API Security

The notification engine exposes multiple API surfaces that must be secured against common attack vectors. Given that the system handles financial data including transaction amounts, portfolio values, and personal identifiers, security is not an afterthought but a core architectural requirement.

A. Authentication & Authorisation

All API endpoints must be protected with JWT-based authentication. Implement role-based access control (RBAC) with at minimum three roles: ADMIN (full system access including DLQ management and analytics), OPERATOR (read-only analytics, DLQ viewing), and SERVICE (machine-to-machine authentication for internal microservices). JWTs must have a maximum TTL of 1 hour with refresh token rotation.

B. Input Validation & Sanitisation

Every API endpoint must validate all inputs against strict schemas using Zod or Joi. Reject requests with unexpected fields (no allowUnknown). For template personalisation data, sanitise all user-provided strings to prevent template injection attacks. Handlebars templates must use triple-brace escaping `{{{field}}}` only when absolutely necessary for HTML content, and double-brace `{{field}}` (which auto-escapes HTML) for all other contexts.

C. Rate Limiting on API Endpoints

Implement per-IP and per-user rate limiting using Redis. Suggested limits: 100 requests/minute for standard API calls, 1000 requests/minute for webhook callbacks (delivery receipts), 10 requests/minute for preference updates. Use sliding window rate limiting (not fixed window) to prevent burst attacks at window boundaries.

D. Secrets Management

All provider API keys, database credentials, and JWT signing keys must be managed through environment variables. Never hardcode secrets. In production, use a secrets manager (AWS Secrets Manager, HashiCorp Vault). For this project, use .env files locally with .env.example committed (no values) and .env in .gitignore.

A10.2 Data Privacy & PII Handling

The notification system processes Personally Identifiable Information (PII) including phone numbers, email addresses, and financial data. Your implementation must:

- Encrypt PII at rest in the database using column-level encryption for phone_number and email fields
- Mask PII in logs: phone numbers displayed as +91XXXX1234, emails as s***@domain.com

- Implement data retention policies: notification content older than 90 days should have personalisation_data scrubbed while retaining metadata for analytics
- Support GDPR-style right-to-erasure: provide an API endpoint that anonymises all notification records for a given user
- Ensure webhook callbacks from providers are verified using signature validation before processing

A10.3 Infrastructure Security

Your Docker Compose configuration must follow security best practices:

- Run application containers as non-root user (USER node in Dockerfile)
- Pin dependency versions in package-lock.json (no floating version ranges in production)
- Configure PostgreSQL with SSL for client connections
- Enable Redis AUTH and disable dangerous commands (FLUSHALL, FLUSHDB, KEYS) in production config
- Use Docker secrets or environment variables for all credentials; never bake credentials into images
- Set resource limits (CPU, memory) in Docker Compose to prevent resource exhaustion



SECTION A11: MONITORING & OBSERVABILITY

A11.1 The Three Pillars of Observability

For a notification engine processing 2M+ messages daily, observability is critical for detecting and diagnosing issues before they impact users. Your system must implement all three pillars:

Pillar 1: Metrics (Prometheus-compatible)

Expose a /metrics endpoint in Prometheus text format. Required metrics:

- notification_events_received_total (counter, labels: event_type, priority)
- notification_delivery_total (counter, labels: channel, provider, status)
- notification_delivery_latency_seconds (histogram, labels: channel, priority)
- notification_frequency_cap_hits_total (counter, labels: cap_type, event_type)
- notification_dnd_blocks_total (counter, labels: classification)
- notification_dlq_depth (gauge)
- notification_retry_total (counter, labels: attempt, provider)
- kafka_consumer_lag (gauge, labels: topic, partition, consumer_group)
- delivery_provider_circuit_state (gauge, labels: provider) — 0=CLOSED, 1=HALF_OPEN, 2=OPEN

Pillar 2: Structured Logging

Every log entry must be a JSON object with at minimum: timestamp (ISO 8601), level (error/warn/info/debug), message, correlation_id (traces notification across services), service_name, and contextual fields relevant to the log event. Log volume management is critical at 2M+ daily messages — use DEBUG level only in development, INFO in staging, WARN in production.

Pillar 3: Distributed Tracing

Implement correlation IDs that follow a notification from event ingestion through to delivery confirmation. The correlation ID must be: generated at event ingestion (UUID v4), propagated through Kafka message headers, included in all log entries for that notification, passed to delivery providers where supported, and stored in the notification record for end-to-end trace reconstruction.

A11.2 Alerting Rules

Define alerting thresholds for critical system health metrics:

Alert Name	Condition	Severity
HighDLQDepth	DLQ depth > 100 for > 5 minutes	CRITICAL - Page on-call engineer
ProviderCircuitOpen	Any provider circuit breaker in OPEN state	HIGH - Notify operations channel
DeliveryLatencySpike	P99 latency > 30 seconds for CRITICAL events	CRITICAL - Page on-call engineer
HighFailureRate	Delivery failure rate > 5% in 10-minute window	HIGH - Notify operations channel
KafkaConsumerLag	Consumer lag > 10,000 messages for > 5 minutes	HIGH - Notify operations channel
FrequencyCapExhaustion	> 50% of users hitting daily cap	MEDIUM - Review notification strategy
DNDViolationDetected	Any DND violation detected	CRITICAL - Page on-call + notify compliance

SECTION A12: SYSTEM DESIGN INTERVIEW PREPARATION

A12.1 Questions the Assessment Panel Will Ask

During the assessment interview, the Customer Retention Board will ask system design questions about your implementation. Prepare detailed answers for the following:

Question 1: How does your system handle a Kafka partition rebalance during peak load?

Expected depth: Explain consumer group rebalancing, what happens to in-flight messages, how your system handles duplicate processing during rebalance, and what consumer configuration parameters you tuned to minimise rebalance duration. Mention `session.timeout.ms`, `heartbeat.interval.ms`, `max.poll.interval.ms`, and `partition.assignment.strategy`.

Question 2: Walk me through exactly what happens when a margin call event enters the system.

Expected depth: Trace the event from Kafka ingestion through enrichment, DND check (explain why transactional margin calls bypass DND), preference resolution (explain regulatory override), template rendering (with personalisation in user's language), channel routing (simultaneous SMS + push + in-app), delivery provider selection, retry on failure, escalation to call queue if all channels fail. Include timing expectations at each stage.

Question 3: How would you scale this system from 2M to 20M notifications daily?

Expected depth: Discuss Kafka partition scaling, consumer group parallelism, database read replicas for analytics queries, Redis Cluster for distributed rate limiting, horizontal scaling of notification workers, and the specific bottlenecks you identified in your load testing that would need to be addressed.

Question 4: What happens if Redis goes down? How does your frequency capping degrade?

Expected depth: Explain your fallback strategy. Options include: bypass capping (risky), use a local in-memory LRU cache with approximate counts (eventual consistency), or fail closed and queue notifications until Redis recovers. Discuss the trade-offs of each approach and which you implemented and why.

Question 5: Show me your database query for generating the daily delivery rate report. What is its execution plan?

Expected depth: Write the actual SQL query, explain the execution plan from EXPLAIN ANALYZE, show how your indexes support the query, explain why you chose the partitioning strategy you did, and estimate query performance at 30-day data volume.

SECTION A13: DELIBERATE ERRORS IN THIS DOCUMENT

This training guide contains five (5) deliberate errors in the technical specifications. Finding and documenting these errors in your README.md under a section titled 'Document Error Log' will earn bonus points. These errors test your attention to detail and domain knowledge. They may be in:

- Event taxonomy definitions (incorrect priority levels, wrong regulatory classifications)
- Database schema definitions (missing constraints, incorrect data types)
- API specification details (incorrect HTTP methods, inconsistent endpoint naming)
- Frequency capping logic (mathematically impossible configurations)
- Retry strategy parameters (configurations that would cause harmful behaviour)



PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

Note on the Simulation Concept: The gamified simulation concept presented in this section is provided as a reference framework. Students may choose to follow this concept or design their own original gamified simulation - with a different name, structure, narrative, or game mechanics - as long as it meets the assessment criteria outlined in Part F.

SECTION B1: SIMULATION SCENARIO - “NOTIFY OR DIE”

B1.1 The Crisis

You are the newly appointed Back-End Lead at WealthBridge, a wealth management app serving 4.2 million users. The app manages ₹32,000 crore in assets and sends 2.1 million notifications daily across SMS, email, push, WhatsApp, and in-app channels.

The board has called an emergency meeting. The metrics are alarming:

- 23.4% opt-out rate (industry average: 8-12%) -the company is haemorrhaging users
- 18.7% of margin calls are delivered AFTER the auto-square-off deadline - causing ₹4.7 crore in customer losses last quarter
- Customer NPS has dropped from +42 to -8 in six months
- TRAI has issued a show-cause notice for 12,847 DND violations in the last 90 days, with a potential fine of ₹50 lakh per violation
- The current monolithic notification system crashes during market volatility, causing 100% notification blackout for 15-45 minutes
- SMS costs have ballooned to ₹18.3 lakh/month due to indiscriminate channel selection

The “Customer Retention Board” (your evaluation panel) has given you 15 days to design and implement a replacement system. If you fail, the company faces regulatory action, customer attrition, and potential licence revocation.

B1.2 The Event Logs

You receive the following production data dumps for analysis (simulated datasets you must create):

- notification_events.csv: 500,000 rows of the last 30 days' notification events with event_type, user_id, channel, status, timestamps, and failure reasons
- user_complaints.csv: 15,000 user complaint records categorised by complaint_type (too_many, irrelevant, wrong_time, wrong_channel, missing_notification)

- `delivery_performance.csv`: Per-provider delivery metrics for the last 90 days showing latency distributions, failure rates, and cost data
- `dnd_violations.csv`: The 12,847 DND violation records with root cause analysis

B1.3 Success Metrics

The Customer Retention Board evaluates your system against these quantified targets:

Metric	Current (Failing)	Target (Your System)
Opt-Out Rate	23.4%	< 8% within 90 days
Critical Notification Latency (P99)	47 seconds	< 10 seconds
DND Violations/Month	4,282	Zero
Delivery Rate (Critical)	81.3%	> 99.5%
System Uptime	97.2% (crashes during volatility)	99.95%
Monthly SMS Cost	₹18.3 lakh	< ₹6 lakh
User Engagement (open rate)	12.4%	> 45%

SECTION B2: ARCHITECTURE CHALLENGE BOARD

B2.1 Challenge 1: The Market Crash Scenario

Simulate a market crash where Nifty drops 8% in 30 minutes. Your system must handle:

- 450,000 price alert notifications triggered simultaneously
- 28,000 margin call notifications with < 10 second delivery requirement
- 3,200 position squared-off notifications
- All while maintaining zero DND violations and respecting quiet hours for non-critical events

Document how your architecture handles this burst without message loss, priority inversion, or cascading failure.

B2.2 Challenge 2: The Provider Outage

Your primary SMS provider (MSG91) goes down at 10:15 AM during a trading session. 35,000 SMS notifications are queued. Your system must:

- Detect the outage within 30 seconds via circuit breaker pattern
- Failover to the secondary SMS provider (Twilio) without message duplication
- Reroute non-critical SMS to WhatsApp as a fallback channel
- Alert operations team via a separate alerting channel
- Resume MSG91 traffic once health check passes, without double-sending

B2.3 Challenge 3: The Compliance Audit

TRAI auditors request a report showing every SMS sent in the last 90 days, with proof that DND status was checked before each send. Your system must produce:

- Audit trail for every notification showing DND check timestamp and result
- Consent records for every user who received promotional messages
- Delivery receipts with timestamps proving margin calls were sent within SEBI timelines
- Complete notification state history for any given notification ID

B2.4 Challenge 4: The Notification Storm

A bug in the market data feed sends 500 duplicate price alert events for the same stock in 10 seconds. Without proper deduplication, $500 \times 150,000$ watching users = 75 million notifications. Your system must:

- Detect and deduplicate the burst within the ingestion layer using event fingerprinting
- Implement idempotency keys based on (event_type + source_entity_id + timestamp_window)
- Ensure only ONE notification per user per stock per event window reaches the routing stage
- Log the deduplication event with metrics: events received, events deduplicated, events passed through
- Generate an automated incident report for the data team showing the duplicate event pattern

Document the exact Redis data structures and TTL configurations you use for deduplication. Explain why your approach prevents both duplicate sends AND missed legitimate events.

B2.5 Challenge 5: The Multi-Language Emergency

RBI announces an emergency rate hike at 2:30 PM on a Tuesday. A regulatory notification (REGX-005) must be sent to all 4.2 million users within 4 hours. The catch: 42% of users prefer Hindi, 18% prefer Marathi, 12% prefer Tamil, 8% prefer Telugu, and 20% prefer English. Your system must:

- Render 4.2 million personalised notifications across 5 languages and 5 channels
- Prioritise delivery to users with active margin positions (they are most affected)
- Respect all frequency caps and quiet hours while still achieving 100% delivery within 4 hours
- Handle template rendering at scale without blocking the critical notification pipeline
- Track per-language delivery rates to ensure no language cohort is systematically delayed

Calculate the throughput requirements: $4.2\text{M notifications} / 4 \text{ hours} = 1,050,000 \text{ per hour} = 17,500 \text{ per minute} = 292 \text{ per second sustained}$. Document how your architecture achieves this throughput.

SECTION B3: SIMULATION SCORING SYSTEM

B3.1 Architecture Score (300 points)

The Customer Retention Board evaluates your architecture against industry best practices:

Architecture Criterion	Points	How Evaluated
Clean separation of concerns (ingestion, routing, delivery, analytics are independent)	60	Code review
Proper use of message brokers (Kafka for streaming, RabbitMQ for routing)	60	Architecture docs
Database design supports query patterns (partitioning, indexing, JSONB)	50	Schema + EXPLAIN
Error handling at every layer with no silent failures	50	Code review
Scalability: system can scale horizontally by adding consumers	40	Load test results
Security: no vulnerabilities in OWASP Top 10 categories	40	Security scan

B3.2 Operational Resilience Score (200 points)

Evaluated through simulated failure injection during assessment:

- Database connection pool exhaustion: does the system degrade gracefully or crash? (50 points)
- Redis unavailability: does frequency capping fail open or fail closed, and is the decision documented? (50 points)
- Kafka broker failure: do in-flight messages survive and resume processing? (50 points)
- SMS provider returning 500 errors: does circuit breaker activate within configured threshold? (50 points)

B3.3 Business Impact Score (200 points)

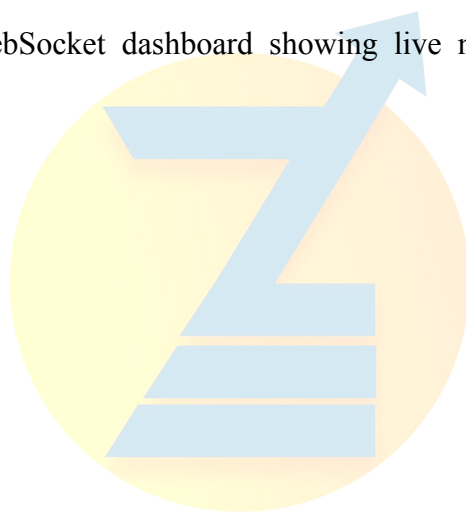
The board measures whether your system actually solves the business problem:

- Opt-out rate reduction: demonstrate through analytics that personalised, well-timed notifications would reduce opt-outs (80 points)
- Cost reduction: show routing intelligence that prefers cheaper channels for non-critical events (60 points)
- Compliance: zero DND violations in your test suite; audit trail completeness (60 points)

B3.4 Bonus Challenges (Up to 100 Bonus Points)

These are optional challenges for interns aiming for A+ scores:

- Implement A/B testing for notification templates with statistical significance calculation (25 bonus points)
- Build a notification preview API that lets users see exactly what they will receive before opting in (25 bonus points)
- Implement intelligent send-time optimisation using per-user historical engagement data (25 bonus points)
- Build a real-time WebSocket dashboard showing live notification metrics (25 bonus points)



ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION B4: SIMULATED DATASET SPECIFICATIONS

B4.1 Datasets You Must Generate

As part of the simulation, you must generate realistic test datasets that your system processes. These datasets will be evaluated for realism and completeness:

Dataset 1: notification_events.csv (Minimum 100,000 rows)

Fields: event_id, event_type (from 25+ taxonomy), user_id, timestamp, priority, source_system, payload (JSON), channel_preference. Distribution: 40% transaction events, 20% risk events, 15% SIP events, 15% market events, 10% regulatory events. Time distribution: 80% during market hours (09:15-15:30 IST), 10% pre-market, 10% post-market.

Dataset 2: user_profiles.csv (Minimum 10,000 rows)

Fields: user_id, name, phone, email, language (en/hi/mr/ta/te), timezone, dnd_status (registered/not_registered), dnd_categories (comma-separated), account_type (basic/premium/HNI), risk_profile (conservative/moderate/aggressive), notification_preferences (JSON), quiet_hours_start, quiet_hours_end, created_at. Distribution: 30% DND registered, 70% not registered. Language: 20% en, 42% hi, 18% mr, 12% ta, 8% te.

Dataset 3: delivery_failures.csv (Minimum 5,000 rows)

Fields: notification_id, channel, provider, failure_code, failure_reason, retry_count, final_status, timestamp. Failure distribution: 35% invalid_recipient, 20% provider_timeout, 15% rate_limited, 10% network_error, 8% invalid_template, 7% dnd_blocked, 5% expired_token.

Dataset 4: user_complaints.csv (Minimum 2,000 rows)

Fields: complaint_id, user_id, complaint_type, channel, event_type, description, timestamp, resolved. Distribution: 35% too_many_notifications, 25% wrong_time (sent during quiet hours), 20% irrelevant_content, 12% wrong_channel, 8% missing_critical_notification.

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

SECTION C1: ZERODHA KITE - HOW INDIA'S LARGEST BROKER HANDLES 10M+ DAILY NOTIFICATIONS

C1.1 Background

Zerodha, India's largest discount broker with 12+ million active users, processes an estimated 10-15 million notifications daily across trading confirmations, margin alerts, and market updates. Their notification architecture evolved from a monolithic system to a distributed event-driven architecture built on Kafka and Redis.

C1.2 Architecture Analysis

Zerodha's publicly documented technology stack reveals several patterns relevant to this project: they use Kafka for event ingestion from their trading engine (Kite Connect), Redis for real-time state management and rate limiting, and a custom Go-based notification service for multi-channel delivery. Their reported system handles 1.5 million concurrent WebSocket connections during peak market hours.

C1.3 Key Lessons for Your Implementation

- Separate transactional notifications (order confirmations) from engagement notifications (market updates) at the architecture level, not just the application level
- Use dedicated Kafka topics per event priority to prevent priority inversion under load
- Implement client-side notification deduplication - network issues can cause duplicate deliveries
- Build a notification preferences UI that is accessible from the trading screen, not buried in settings

C1.4 Simulation Scenario

Design a test scenario that replicates Zerodha's peak load: 150,000 simultaneous order executions triggering multi-channel notifications, with 95th percentile delivery within 5 seconds. Document your approach to handling this load.

SECTION C2: WHATSAPP BUSINESS API RATE LIMITING - THE PAYTM MONEY CASE

C2.1 Background

In 2023, several Indian fintech companies including Paytm Money faced challenges with WhatsApp Business API rate limiting. WhatsApp enforces tier-based rate limits: new business accounts start at Tier 1 (1,000 unique users/day) and can scale to Tier 4 (100,000 unique users/day) based on quality rating. Companies that exceeded their tier limits or had poor quality scores saw their API access throttled or suspended.

C2.2 Technical Analysis

The rate limiting mechanism operates on multiple dimensions: per-phone-number throughput (80 messages/second), per-business-account daily unique user limits, and quality-based throttling. When a business's quality rating drops (due to user blocks or spam reports), WhatsApp automatically downgrades the tier, creating a cascading failure where notification backlogs grow while capacity shrinks.

C2.3 Key Lessons

- Implement client-side rate limiting that mirrors provider limits to avoid hitting API limits
- Monitor quality metrics (delivery rate, read rate, block rate) per WhatsApp template
- Maintain a warm-up strategy for new WhatsApp business numbers
- Have fallback channels ready - if WhatsApp is throttled, reroute to SMS or push

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTOIN C3: TRAI DND ENFORCEMENT - THE ₹20 CRORE FINE WAVE

C3.1 Background

In 2022-2023, TRAI imposed penalties exceeding ₹20 crore on telecom operators and principal entities for DND violations. Financial services companies were among the top violators because their systems did not adequately distinguish between transactional and promotional messages, and DND scrubbing databases were not updated frequently enough.

C3.2 Root Cause Analysis

The primary causes of DND violations in financial services were: stale DND databases (updated weekly instead of daily), incorrect message categorisation (promotional messages tagged as transactional to bypass DND), missing consent records for users who had opted into specific categories, and race conditions where a user registered for DND between the scrubbing check and actual SMS delivery.

C3.3 Key Lessons

- Refresh DND database daily minimum, preferably in real-time via TRAI API
- Implement a DND check at the LAST possible moment before SMS dispatch, not during routing
- Maintain immutable consent audit logs with timestamps, IP addresses, and consent text
- Classify every notification template as TRANSACTIONAL or PROMOTIONAL during template creation, with human review for borderline cases
- Implement automated DND violation detection and self-reporting mechanisms

SECTION C4: ROBINHOOD'S PUSH NOTIFICATION OUTAGE DURING THE GAMESTOP SAGA

C4.1 Background

During the January 2021 GameStop (GME) trading frenzy, Robinhood's notification system experienced a catastrophic failure. Users reported not receiving trade execution confirmations, margin call warnings, or account restriction notifications. Some users discovered their positions had been liquidated only hours later when they manually checked their accounts. The backlash contributed to Robinhood's \$70M FINRA fine for causing significant harm to millions of customers.

C4.2 Technical Analysis

The failure cascaded from an overwhelmed event processing pipeline. The notification service was not designed for the 10-20x traffic spike. Without proper backpressure mechanisms, the event queue filled up, causing the service to drop notifications. Critical margin call notifications were treated with the same priority as informational notifications, leading to priority inversion where users received price alerts but not margin calls.

C4.3 Key Lessons

- CRITICAL notifications MUST be on a separate processing pipeline from non-critical ones - not just separate queues, but separate consumer groups with dedicated resources
- Implement backpressure: when the system is overloaded, shed LOW-priority notifications first
- For regulatory-required notifications (margin calls), implement a guaranteed delivery mechanism with fallback to SMS and human escalation
- Load test at 20x expected peak, not just 2-3x

SECTION C5: GROWW'S NOTIFICATION PERSONALISATION ENGINE - REDUCING OPT-OUTS FROM 18% TO 6%

C5.1 Background

Groww, India's fastest-growing investment platform, faced a notification opt-out rate of approximately 18% in 2022. Their analysis revealed that the primary driver was not notification volume, but notification irrelevance. Users investing only in mutual funds were receiving push notifications about stock price movements. Users with conservative risk profiles were receiving derivatives trading alerts. The one-size-fits-all approach was driving away the very users the platform wanted to retain.

C5.2 Technical Solution

Groww implemented a three-tier personalisation strategy: first, behavioural segmentation based on actual user activity (trading frequency, product usage, session patterns); second, explicit preference capture through an onboarding flow that asked users what notifications they wanted; third, machine learning-based send-time optimisation that delivered notifications when individual users were most likely to engage. The system maintained per-user engagement profiles updated in real-time based on notification opens, clicks, and dismissals.

C5.3 Architecture Patterns

The personalisation engine used a feature store (Redis) that maintained pre-computed user feature vectors including: last 7-day activity pattern, preferred channels (based on historical engagement), preferred notification time windows, product affinity scores, and risk tolerance indicators. These features were computed in batch (daily) and updated incrementally (on each user action). The routing engine queried the feature store at routing time (< 2ms latency) to make personalisation decisions.

C5.4 Key Lessons

- Personalisation must go beyond "insert user name" - it means sending the RIGHT content on the RIGHT channel at the RIGHT time
- Build a feedback loop: track which notifications users engage with and use that data to improve future routing
- Allow users to self-select notification categories during onboarding, but also infer from behaviour
- The feature store pattern (pre-compute, cache in Redis, query at routing time) achieves sub-millisecond personalisation at scale
- A/B test notification strategies before rolling out: test on 5% of users, measure engagement, then scale

C5.5 Simulation Scenario

Using the user_profiles.csv dataset you generate, build a notification routing simulation that demonstrates personalised channel selection. For a given MKTX-004 (52-Week High) event, show how the routing engine makes different channel decisions for three user archetypes: an active trader (prefers push, checks app 15x/day), a passive SIP investor (prefers weekly email digest), and a dormant user (hasn't logged in for 30 days, route to WhatsApp for re-engagement). Document the scoring algorithm that drives these decisions.



SECTION C6: SLACK'S NOTIFICATION INFRASTRUCTURE - LESSONS FROM PROCESSING 5 BILLION DAILY EVENTS

C6.1 Background

Slack processes over 5 billion events daily across its notification infrastructure, delivering messages across push, email digest, desktop, and in-app channels. While Slack's domain is workplace messaging rather than financial services, its notification architecture offers directly transferable patterns for high-throughput event processing, preference management, and delivery optimisation.

C6.2 Architecture Patterns

Slack's public engineering blog reveals several architectural patterns relevant to this project. Their notification system uses a two-phase processing model: first, an event qualification phase that determines IF a notification should be sent (based on user presence, mute settings, channel notification preferences, and DND status); second, a delivery phase that determines HOW to deliver (push vs. email digest vs. desktop notification). This separation ensures that the expensive delivery phase only processes qualified events.

Their preference system operates at five levels: workspace-level defaults, channel-level overrides, thread-level overrides, keyword-based overrides, and per-device settings. This hierarchical preference model directly parallels the four-layer hierarchy described in Part A Section A5.1 of this document. The key insight is that preference resolution must be fast (cached) and deterministic (same inputs always produce same routing decision, which is critical for debugging).

C6.3 The "Notification Budget" Concept

Slack introduced the concept of a "notification budget" - each user has a finite tolerance for notifications per day, and every notification "spends" from that budget. Critical notifications (direct messages, @mentions) have zero cost against the budget. Channel activity notifications have increasing cost based on frequency. When the budget is exhausted, lower-priority notifications are silently converted to batch/digest delivery. This concept maps directly to frequency capping in your system.

C6.4 Key Lessons

- Two-phase processing (qualify THEN deliver) reduces load on the expensive delivery pipeline
- Preference hierarchies must be cached and resolution must be deterministic for debugging
- The "notification budget" concept is more user-friendly than hard frequency caps - users feel in control
- Separate notification state (has the user been notified?) from delivery state (was it delivered?) for accurate metrics

- Use dead-letter tracking not just for failed deliveries, but for suppressed notifications — this data reveals over-notification patterns



ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION C7: COMPARATIVE ANALYSIS - NOTIFICATION ARCHITECTURE PATTERNS ACROSS INDIAN FINTECH

C7.1 Architecture Comparison Matrix

Based on publicly available engineering blogs, conference talks, and technical documentation, the following comparison captures notification architecture patterns across major Indian financial platforms:

Dimension	Zerodha	Groww	Paytm Money	Angel One	Your System
Primary Event Bus	Kafka	Kafka	RabbitMQ	Kafka	Kafka + RabbitMQ
Cache Layer	Redis	Redis	Redis	Redis	Redis
Template Engine	Custom Go	Jinja2	Handlebars	Custom Java	Handlebars
Personalisation	Basic (name)	ML-based	Segment-based	Rule-based	Hybrid (rules + data)
Languages Supported	English + Hindi	5 languages	3 languages	English + Hindi	5 languages
Frequency Capping	Per-channel daily	Multi-dimensional	Global daily	Per-channel	Multi-dimensional
DND Compliance	Batch scrubbing	Real-time check	Batch + real-time	Batch scrubbing	Real-time at dispatch
Circuit Breaker	Yes (Go built-in)	Yes (Hystrix)	Partial	Yes	Yes (custom)
Priority Queues	Separate topics	Separate topics	RabbitMQ priority	Separate topics	Both approaches

The "Your System" column represents the target architecture specified in this document. Note that your system combines best practices from multiple platforms: Kafka for event streaming (like Zerodha), multi-dimensional frequency capping (like Groww), real-time DND checking at dispatch (like Groww), and RabbitMQ priority queues for delivery routing.

C7.2 Anti-Patterns to Avoid

Analysis of production failures across the industry reveals common anti-patterns that your implementation MUST avoid:

Anti-Pattern	Description	Consequence
The Fire Hose	Sending every event as a notification without qualification or filtering. No distinction between informational and actionable events.	23%+ opt-out rate, user fatigue, brand damage
Synchronous Delivery	Making delivery API calls synchronously in the event processing pipeline. A slow provider blocks all notifications.	Cascading latency, priority inversion, missed SLAs
Shared Queue Hell	Processing CRITICAL and LOW priority notifications in the same queue with same consumers. Head-of-line blocking.	Margin calls delayed behind market updates
Stale DND Cache	Checking DND status at routing time (minutes before send) instead of at dispatch time (milliseconds before send).	DND violations, regulatory fines, user complaints
Template Monolith	Hardcoding notification content in application code. Changes require code deployment.	Slow iteration, inconsistent messaging, deployment risk
Retry Storm	Retrying immediately and indefinitely without backoff or cap. When a provider recovers, it gets hit with a thundering herd.	Provider rate limiting, double sends, extended outage
Silent Failure	Swallowing errors with catch blocks that only log. No alerting, no DLQ, no retry.	Lost notifications, undetected failures, data inconsistency
Log Blindness	Logging without correlation IDs. Cannot trace a notification from ingestion to delivery.	Hours spent debugging, cannot answer compliance queries

ADD EXPERIENCE TO KNOWLEDGE

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

Day 1 - Project Setup & Architecture Design:

Task:

- Create PRIVATE GitHub repository: BE-6B-NotificationEngine-[YourName]
- Initialise [Node.js](#)/TypeScript project with strict TypeScript configuration (strict: true, noImplicitAny: true)
- Set up project structure: src/, tests/, docs/, config/, scripts/, migrations/
- Configure ESLint with @typescript-eslint/recommended and Prettier
- Set up Docker Compose with: PostgreSQL 15, Redis 7, Kafka (Confluent), RabbitMQ 3.12
- Create .env.example with all required environment variables (NEVER commit .env)
- Write initial README.md with project overview and architecture decision records
- Create docs/architecture.md with: system context diagram, container diagram, component diagram (C4 model)
- Document technology choices with justification for each decision
- Design the event processing pipeline from ingestion to delivery
- Define API contracts for all internal services using OpenAPI 3.0 specification
- Create docs/event-taxonomy.yaml with all 25+ event type definitions

Deliverable:

- Git commit: 'feat: initial project setup with Docker Compose and architecture docs'
- Working Docker Compose that starts all infrastructure services
- Architecture design document with diagrams
- Event taxonomy YAML with all 25+ event definitions

Day 2 - Database Schema & Event Models:

Task:

- Implement database migrations using a migration tool ([Knex.js](#), Prisma, or TypeORM)
- Create all core tables: users, notifications, notification_state_log, dead_letter_queue, user_preferences, templates, delivery_providers, consent_records
- Implement table partitioning for notifications table by created_at (monthly partitions)
- Create all indexes as specified in Part A Section A8.2
- Write seed data script with 1,000 test users with varied preferences
- Define TypeScript interfaces for all 25+ event types with strict typing
- Implement JSON Schema or Zod validation for all event payloads
- Create event factory pattern for generating test events
- Write unit tests for all event validators (target: 100% coverage of validation logic)

Deliverable:

- Git commit: 'feat: database schema, migrations, event models, and validation'
- Passing migration scripts that create all tables and indexes
- TypeScript event models with strict typing
- Unit tests for event validation (minimum 20 test cases)

Day 3 - Event Ingestion Pipeline:

Task:

- Configure Kafka topics: notification-events (partitioned by user_id hash), notification-critical (high-priority dedicated topic), notification-dlq
- Implement Kafka producer with idempotent configuration (enable.idempotence=true)
- Implement consumer groups with manual offset management for at-least-once delivery
- Add event serialisation with Avro/JSON Schema for schema evolution support
- Implement event enrichment: resolve user context, apply preference hierarchy, determine channels
- Build the event routing engine that maps events to delivery channels based on the decision matrix from Part A Section A3.2

- Implement event deduplication using Redis with TTL-based idempotency keys
- Write integration tests for the ingestion pipeline (produce event → consume → route)

Deliverable:

- Git commit: 'feat: Kafka-based event ingestion pipeline with routing engine'
- Working Kafka producer/consumer with proper configuration
- Event enrichment and routing logic
- Integration tests for the pipeline

Day 4 - Template Engine & Personalisation:

Task:

- Implement Handlebars-based template engine with custom helpers for currency formatting, date formatting, and truncation
- Create template registry that loads templates from database/file system
- Implement template versioning with A/B testing support
- Build template validation that checks all required personalisation fields are present
- Implement personalisation pipeline: resolve user context → compute derived fields → format values → render template
- Add localisation support for English, Hindi, Marathi, Tamil, Telugu
- Implement SMS truncation logic that preserves meaning within 160 chars
- Create templates for all 25+ event types across all channels
- Write unit tests for template rendering with edge cases (missing fields, long names, special characters)

Deliverable:

- Git commit: 'feat: template engine with personalisation and localisation'
- Working template engine with all 25+ event templates
- Localisation files for 5 languages
- Unit tests for template rendering (minimum 30 test cases)

Day 5 - User Preference System:

Task:

- Implement RESTful API for user preferences: GET /users/:id/preferences, PUT /users/:id/preferences
- Build preference hierarchy resolver (system defaults → segment → user → regulatory override)
- Implement preference caching in Redis with cache invalidation on update
- Add preference migration logic for existing users (apply default preferences on first access)
- Integrate preference system with the routing engine from Day 3
- Implement digest mode: batch low-priority notifications and deliver as a single digest
- Build preference analytics: track which preferences users change most frequently
- Write API tests for all preference endpoints

Deliverable:

- Git commit: 'feat: user preference system with caching and routing integration'
- Working preference API with Redis caching
- Preference hierarchy resolver
- API tests for preference endpoints

Day 6 - DND Compliance & Frequency Capping:

Task:

- Implement DND registry lookup service with local cache (simulated DND database)
- Build DND classification engine: classify every notification as TRANSACTIONAL or PROMOTIONAL
- Add consent management: record opt-in/opt-out with immutable audit log
- Implement the DND check as the LAST step before SMS dispatch
- Implement multi-dimensional frequency capping using Redis atomic operations
- Build quiet hours enforcement with per-user timezone resolution
- Implement quiet hours queue that aggregates notifications into morning digests

- Add CRITICAL event bypass logic with audit trail
- Write comprehensive tests for DND and frequency capping (minimum 25 test cases covering all edge cases)

Deliverable:

- Git commit: 'feat: DND compliance, consent management, frequency capping, quiet hours'
- DND lookup service with caching
- Frequency capping with Redis atomics
- Quiet hours enforcement
- Test suite with 25+ test cases

Day 7 - Multi-Channel Delivery Providers:

Task:

- Define the DeliveryProvider interface as specified in Part A Section A3.3
- Implement SMS provider (MSG91 sandbox or Twilio test mode)
- Implement email provider using Nodemailer with Ethereal test SMTP
- Add provider health check mechanism
- Implement push notification provider using FCM HTTP v1 API (test mode)
- Implement WhatsApp provider using WhatsApp Cloud API test mode
- Implement in-app notification provider using WebSocket (Socket.io)
- Add provider-level rate limiting and circuit breaking
- Write provider integration tests with mock servers

Deliverable:

- Git commit: 'feat: multi-channel delivery providers with circuit breaking'
- 5 delivery provider implementations (SMS, email, push, WhatsApp, in-app)
- Circuit breaker pattern implementation
- Integration tests for each provider

Day 8 - Delivery Routing & Failover Engine:

Task:

- Implement the priority-weighted channel scoring model from Part A Section A3.2
- Build multi-channel fan-out for events requiring simultaneous delivery
- Add channel failover logic: if primary channel fails, route to next-best channel
- Implement delivery acknowledgement tracking with provider callbacks
- Implement circuit breaker pattern with three states: CLOSED, OPEN, HALF-OPEN
- Configure circuit breaker thresholds per provider (failure rate, response time)
- Build provider failover: MSG91 fails → Twilio; FCM fails → APNS
- Add idempotency checks to prevent duplicate delivery during failover
- Write failover simulation tests

Deliverable:

- Git commit: 'feat: intelligent routing engine with failover and circuit breaking'
- Channel scoring and routing engine
- Provider failover with circuit breaker
- Failover simulation test suite

Day 9 - Retry Strategy & Dead Letter Queue:

Task:

- Implement exponential backoff retry worker with jitter as specified in Part A Section A9
- Configure priority-based retry policies (CRITICAL: 10 retries, LOW: 2 retries)
- Build retry scheduling using Redis sorted sets (ZADD with retry timestamp as score)
- Add retry budget monitoring to prevent retry storms
- Implement DLQ consumer that processes failed notifications after max retries
- Build DLQ dashboard API: list DLQ entries, filter by reason, manually retry or discard
- Add automated DLQ classification: categorise failures as transient, permanent, or configuration error
- Implement DLQ alerting: if DLQ depth exceeds threshold, alert operations

- Write DLQ processing tests

Deliverable:

- Git commit: 'feat: retry strategy with exponential backoff and DLQ processing'
- Retry worker with configurable policies
- DLQ processing pipeline with classification
- DLQ management API

Day 10 - Analytics Pipeline:

Task:

- Implement real-time counters using Redis: delivery counts, failure counts, latency tracking per channel
- Build sliding window analytics for hourly/daily/weekly aggregations
- Add Prometheus-compatible metrics endpoint (/metrics) for monitoring integration
- Implement event-sourced analytics: derive all metrics from the notification_state_log
- Build analytics API endpoints: GET /analytics/delivery-rates, GET /analytics/channel-performance, GET /analytics/opt-out-trends
- Implement time-series aggregation queries optimised with the BRIN indexes
- Add cost analytics: per-channel, per-event-type, per-user cost tracking
- Create mock dashboard data generation for simulation datasets

Deliverable:

- Git commit: 'feat: real-time analytics pipeline with metrics and API'
- Real-time metrics with Redis
- Analytics API endpoints
- Prometheus-compatible metrics

Day 11 - Load Testing & Performance Optimisation:

Task:

- Set up k6 or Artillery for load testing
- Create load test scenarios: normal load (2M messages/day), peak load (10x), market crash (20x)

- Test the market crash scenario from Part B: 450K notifications in 30 minutes
- Measure and document P50, P95, P99 latencies under each load scenario
- Profile and optimise hot paths based on load test results
- Implement connection pooling for PostgreSQL and Redis
- Add Kafka consumer parallelism tuning
- Optimise database queries (use EXPLAIN ANALYZE, add missing indexes)
- Document performance benchmarks and bottleneck analysis

Deliverable:

- Git commit: 'feat: load testing suite and performance optimisations'
- Load test scripts for all scenarios
- Performance benchmark report with P50/P95/P99 latencies
- Query optimisation evidence (EXPLAIN ANALYZE outputs)

Day 12 - Error Handling, Logging & Monitoring:

Task:

- Implement structured JSON logging with Winston or Pino
- Add correlation IDs that trace a notification from event ingestion to delivery
- Configure log levels: ERROR, WARN, INFO, DEBUG with environment-based filtering
- Add request logging middleware for all API endpoints
- Implement global error handler with error classification (transient, permanent, validation)
- Build comprehensive health check endpoint (/health) that checks: database, Redis, Kafka, RabbitMQ, all providers
- Add readiness and liveness probes for container orchestration
- Implement graceful shutdown: drain queues, complete in-flight notifications, close connections
- Add alerting rules for critical failures

Deliverable:

- Git commit: 'feat: structured logging, error handling, health checks, graceful shutdown'
- Structured logging with correlation IDs
- Health check endpoints
- Graceful shutdown implementation

Day 13 - API Documentation & Comprehensive Testing:

Task:

- Generate OpenAPI 3.0 specification for all API endpoints
- Set up Swagger UI at /api-docs for interactive API exploration
- Document all request/response schemas with examples
- Create Postman collection with pre-configured test requests
- Achieve minimum 80% code coverage across the entire codebase
- Write end-to-end tests: event ingestion → processing → routing → delivery → tracking
- Write edge case tests: DND registered user receives mandatory margin call, user changes preference mid-delivery, provider switches during failover
- Run mutation testing to validate test quality (if time permits)

Deliverable:

- Git commit: 'feat: OpenAPI docs, Swagger UI, comprehensive test suite'
- OpenAPI specification and Swagger UI
- Postman collection
- Test coverage report showing > 80% coverage

Day 14 - Containerisation, CI/CD & Security:

Task:

- Create multi-stage Dockerfile for the application (build → production)
- Optimise Docker image size (target < 200MB)
- Update Docker Compose with all services, health checks, and resource limits
- Create docker-compose.test.yml for running tests in CI

- Set up GitHub Actions CI pipeline: lint → test → build → coverage report
- Add security scanning with npm audit and Snyk (or similar)
- Implement rate limiting on all public API endpoints
- Add input sanitisation and SQL injection prevention
- Verify no secrets in codebase (use git-secrets or similar)

Deliverable:

- Git commit: 'feat: Docker, CI/CD pipeline, security hardening'
- Multi-stage Dockerfile
- GitHub Actions CI pipeline
- Security scan results

Day 15 - Final Documentation & Repository Transfer:

Task:

- Complete README.md with: project overview, architecture diagrams, setup instructions, API documentation links, test running instructions, deployment guide
- Create ARCHITECTURE.md with: C4 diagrams, sequence diagrams for key flows, database ER diagram
- Create CHANGELOG.md with daily progress log
- Write DEPLOYMENT.md with production deployment checklist
- Ensure all code is properly commented with JSDoc/TSDoc
- Run full test suite and ensure 100% pass rate
- Run linter and fix any remaining warnings
- Verify Docker Compose starts all services successfully
- Remove any test credentials, debugging code, or TODO comments
- Add @ZethetaIntern as a collaborator
- Transfer repository ownership to @ZethetaIntern
- Verify transfer was successful and all branches/history are intact

Deliverable:

- Final git commit: 'chore: final documentation and cleanup for repository transfer'
- Complete README.md with all sections
- Architecture documentation with diagrams
- Successful repository transfer to @ZethetaIntern

Submission Instructions:

- All files to be named using convention: ProjectCode_YourName_DeliverableName (e.g., 201169C_RahulShah_MainReport)
- All PDFs must be readable and not password-protected
- Supporting Excel files must be in .xlsx format
- All daily deliverables across Days 1–15 are to be submitted on **Day 15** as a consolidated final submission. Students should combine their work into **one or two PDFs and/or one Excel file** wherever possible, rather than submitting separate files for each day. Clearly label each section within the consolidated document by its respective day and topic for ease of assessment.

Late submissions attract the penalties specified in Part F Assessment Criteria

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART E: NO-CODE TOOLS & PLATFORMS GUIDE

SECTION E1: REQUIRED TECHNOLOGY STACK

Category	Required Tool	Purpose
Runtime	Node.js 20 LTS + TypeScript 5.x	Server-side runtime with strict typing
Framework	Express.js or Fastify	HTTP API framework (Fastify preferred for performance)
Database	PostgreSQL 15+	Primary data store with partitioning and JSONB support
Cache / State	Redis 7+	Rate limiting, caching, frequency caps, sorted set queues
Event Streaming	Apache Kafka (Confluent)	Event bus for ingestion, fan-out, and replay
Message Queue	RabbitMQ 3.12+	Delivery routing with priority queues and DLQ
ORM / Query Builder	Prisma or Knex.js	Type-safe database access with migrations
SMS Provider	MSG91 / Twilio (sandbox)	SMS delivery with DLR callbacks
Email Provider	Nodemailer + Ethereal	Email delivery with HTML templates
Push Provider	FCM (Firebase Cloud Messaging)	Push notification delivery
Template Engine	Handlebars.js	Template rendering with helpers
Testing	Jest + Supertest	Unit, integration, and API testing
Load Testing	k6 or Artillery	Performance and stress testing
Containerisation	Docker + Docker Compose	Local development and CI environments
CI/CD	GitHub Actions	Automated testing and build pipeline
Documentation	OpenAPI 3.0 + Swagger UI	API documentation
Diagramming	Draw.io / Mermaid	Architecture and sequence diagrams
DB Design	dbdiagram.io	ER diagram and schema documentation

SECTION E2: REPOSITORY STRUCTURE

Your repository **MUST** follow this exact structure. The automated assessment tool expects these paths:

```
BE-6B-NotificationEngine-[YourName] /
├── .github/
│   └── workflows/
│       └── ci.yml
├── docs/
│   ├── architecture.md
│   ├── api-specification.yaml
│   ├── event-taxonomy.yaml
│   ├── database-schema.md
│   ├── sequence-diagrams/
│   └── performance-benchmarks.md
├── src/
│   ├── config/
│   ├── events/
│   │   ├── models/
│   │   ├── validators/
│   │   └── taxonomy.yaml
│   ├── notifications/
│   │   ├── engine/
│   │   ├── routing/
│   │   └── state-machine/
│   ├── delivery/
│   │   ├── providers/
│   │   ├── circuit-breaker/
│   │   └── retry/
│   ├── templates/
│   │   ├── engine/
│   │   ├── definitions/
│   │   └── locales/
│   ├── preferences/
│   ├── compliance/
│   └── dnd/
```

```

├── frequency-cap/
│   └── quiet-hours/
├── analytics/
├── api/
│   ├── routes/
│   ├── middleware/
│   └── validators/
├── database/
│   ├── migrations/
│   ├── seeds/
│   └── models/
├── utils/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   └── load/
├── templates/      (Handlebars template files)
├── docker-compose.yml
├── docker-compose.test.yml
├── Dockerfile
├── .env.example
├── .zetheta-project.json
├── tsconfig.json
├── jest.config.ts
├── package.json
├── README.md
├── ARCHITECTURE.md
├── CHANGELOG.md
└── DEPLOYMENT.md

```

SECTION E3: MANDATORY.zetheta-project.json

```
{
  "intern_id": "[YOUR_INTERN_ID]",
  "intern_name": "[YOUR_NAME]",
  "intern_email": "[YOUR_EMAIL]",
  "project_code": "BE-6B",
  "project_title": "Event-Driven Notification Engine with
Multi-Channel Delivery",
  "submission_type": "github",
  "tech_stack": ["TypeScript", "Node.js", "PostgreSQL",
"Redis", "Kafka", "RabbitMQ", "Docker"],
  "duration_days": 15,
  "github_repo_url":
"https://github.com/[YOUR_USERNAME]/BE-6B-NotificationEngine-[
YourName]",
  "assessment_criteria": {
    "event_driven_architecture": true,
    "multi_channel_delivery": true,
    "template_engine": true,
    "user_preferences": true,
    "dnd_compliance": true,
    "frequency_capping": true,
    "delivery_tracking": true,
    "dead_letter_queue": true,
    "analytics_pipeline": true,
    "load_testing": true,
    "ci_cd_pipeline": true
  }
}
```

SECTION E4: AI-ACCELERATED DEVELOPMENT BEST PRACTICES

You are expected to use AI tools (Claude, Copilot, ChatGPT) to accelerate development. However, blind copy-paste is prohibited. Follow these guidelines:

- Use AI for boilerplate code generation (Docker configs, CI pipelines, test scaffolding) but review every line
- Use AI to explain complex concepts (Kafka consumer groups, circuit breaker state machines) but implement in your own code
- Use AI for code review: paste your implementation and ask for security vulnerabilities, performance issues, and edge cases
- Document AI usage: in your `CHANGELOG.md`, note where AI significantly accelerated a task
- NEVER commit AI-generated code you cannot explain line-by-line if asked during assessment



SECTION E5: REFERENCE DOCUMENTATION

Study these resources before and during the project:

- Apache Kafka Documentation: <https://kafka.apache.org/documentation/> - Focus on: Producer API, Consumer Groups, Partitioning Strategy
- RabbitMQ Tutorials: <https://www.rabbitmq.com/tutorials> - Focus on: Priority Queues, Dead Letter Exchanges, Publisher Confirms
- Redis Documentation: <https://redis.io/docs/> - Focus on: Sorted Sets, Streams, Lua Scripting, Rate Limiting patterns
- MSG91 API Docs: <https://docs.msg91.com/> - SMS delivery and DLR callbacks
- Twilio API Docs: <https://www.twilio.com/docs/sms> - Alternative SMS provider
- FCM HTTP v1 API: <https://firebase.google.com/docs/cloud-messaging> - Push notification delivery
- WhatsApp Cloud API: <https://developers.facebook.com/docs/whatsapp/cloud-api> - WhatsApp Business messaging
- TRAI DND Regulations: <https://traigov.in> - DND compliance requirements
- Martin Fowler - Circuit Breaker Pattern: <https://martinfowler.com/bliki/CircuitBreaker.html>
- Enterprise Integration Patterns (Hohpe & Woolf) - Messaging patterns reference

PART F: SUBMISSION PROTOCOL

Projects are assessed using a comprehensive 1000-point rubric covering seven dimensions: Problem Understanding, Solution Quality, Research & Analysis, Presentation & Clarity, Innovation & Creativity, Feasibility & Practicality and CV alignment. Assessment is conducted through a combination of manual review and AI-powered evaluation.



APPENDIX A: SAMPLE API REQUEST/RESPONSE PAYLOADS

This appendix provides exact API contract specifications that your implementation must conform to. The automated assessment tool will validate these contracts.

Event Ingestion API

POST /api/v1/events

Accepts a new financial event for notification processing. This is the primary entry point for all event producers (trading engine, market data feed, compliance system, etc.).

Request Body:

```
{
  "event_type": "RISK-001",
  "event_id": "EVT-2025-03-19-MC-847291",
  "source_system": "margin_engine",
  "timestamp": "2025-03-19T10:15:23.456Z",
  "priority": 1,
  "user_id": "usr_alb2c3d4-e5f6-7890-abcd-ef1234567890",
  "payload": {
    "shortfall_amount": 125000.00,
    "current_margin": 375000.00,
    "required_margin": 500000.00,
    "deadline": "2025-03-19T11:30:00.000Z",
    "affected_positions": [
      { "symbol": "RELIANCE", "qty": 500, "current_value":
620000.00 },
      { "symbol": "INFY", "qty": 200, "current_value":
340000.00 }
    ],
    "auto_square_off_time": "2025-03-19T12:00:00.000Z"
  },
  "idempotency_key":
"margin-call-usr_alb2c3d4-2025-03-19T10:15"
}
```

Success Response (202 Accepted):

```
{
  "notification_id":
  "ntf_98765432-abcd-1234-ef56-789012345678",
  "event_id": "EVT-2025-03-19-MC-847291",
  "status": "CREATED",
  "channels_targeted": ["sms", "push", "in_app"],
  "estimated_delivery_ms": 3000,
  "created_at": "2025-03-19T10:15:23.512Z"
}
```

Error Response (422 Unprocessable Entity):

```
{
  "error": "VALIDATION_FAILED",
  "message": "Event payload validation failed",
  "details": [
    { "field": "payload.shortfall_amount", "error": "must be a
positive number" },
    { "field": "payload.deadline", "error": "must be a future
timestamp" }
  ],
  "request_id": "req_abc123def456"
}
```

User Preferences API**GET /api/v1/users/:userId/preferences****Response (200 OK):**

```
{
  "user_id": "usr_a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "global_preferences": {
    "quiet_hours": { "start": "22:00", "end": "07:30",
"timezone": "Asia/Kolkata" },
    "language": "hi",
    "digest_mode": "none"
  },
}
```

```
"category_preferences": [
  {
    "category": "TXNX",
    "channels": { "sms": true, "email": true, "push": true,
"whatsapp": false, "in_app": true },
    "digest_mode": "immediate"
  },
  {
    "category": "MKTX",
    "channels": { "sms": false, "email": true, "push": true,
"whatsapp": false, "in_app": true },
    "digest_mode": "daily"
  },
  {
    "category": "SIPX",
    "channels": { "sms": true, "email": true, "push": true,
"whatsapp": true, "in_app": true },
    "digest_mode": "immediate"
  }
],
"regulatory_overrides": [
  { "event_type": "RISK-001", "channels": ["sms", "push"],
"cannot_disable": true },
  { "event_type": "RISK-002", "channels": ["sms", "push",
"email"], "cannot_disable": true },
  { "event_type": "REGX-001", "channels": ["sms", "email"],
"cannot_disable": true }
],
"updated_at": "2025-03-15T14:22:00.000Z"
}
```

PUT /api/v1/users/:userId/preferences

Request Body:

```
{
  "category": "MKTX",
  "channels": { "sms": false, "email": false, "push": true,
"whatsapp": false, "in_app": true },
  "digest_mode": "daily"
}
```

Response (200 OK):

```
{
  "status": "updated",
  "category": "MKTX",
  "warnings": [
    "Email disabled for MKTX – you will not receive earnings announcements (MKTX-005) via email. Regulatory notifications in this category will still be delivered via mandatory channels."
  ],
  "cache_invalidated": true,
  "updated_at": "2025-03-19T10:30:00.000Z"
}
```

Analytics API**GET /api/v1/analytics/delivery-rates?period=7d&channel=all****Response (200 OK):**

```
{
  "period": { "start": "2025-03-12T00:00:00Z", "end": "2025-03-19T00:00:00Z" },
  "summary": {
    "total_sent": 14823456,
    "total_delivered": 14156892,
    "total_read": 6570189,
    "overall_delivery_rate": 0.9550,
    "overall_read_rate": 0.4431
  },
  "by_channel": {
    "sms": { "sent": 3240000, "delivered": 3110400, "rate": 0.9600 },
    "email": { "sent": 2890000, "delivered": 2601000, "rate": 0.9000 },
    "push": { "sent": 4560000, "delivered": 3420000, "rate": 0.7500 },
    "whatsapp": { "sent": 2133456, "delivered": 2026783, "rate": 0.9500 },
    "in_app": { "sent": 2000000, "delivered": 1998709, "rate": 0.9994 }
  }
}
```

```
},
  "by_priority": {
    "CRITICAL": { "sent": 156000, "delivered": 155844, "rate":
0.9990, "p99_latency_ms": 4200 },
    "HIGH": { "sent": 5890000, "delivered": 5654400, "rate":
0.9600, "p99_latency_ms": 12500 },
    "MEDIUM": { "sent": 4560000, "delivered": 4286400, "rate":
0.9400, "p99_latency_ms": 45000 },
    "LOW": { "sent": 4217456, "delivered": 4060248, "rate":
0.9627, "p99_latency_ms": 180000 }
  },
  "frequency_cap_hits": { "total": 234567,
"percentage_of_users_hitting_cap": 0.12 },
  "dnd_blocks": { "total": 89234, "promotional_blocked":
87891, "transactional_blocked": 0 },
  "cost_summary": { "total_inr": 423567,
"per_notification_avg_paisa": 2.86 }
}
```

Notification Status API

GET /api/v1/notifications/:notificationId

Response (200 OK):

```
{
  "notification_id":
"ntf_98765432-abcd-1234-ef56-789012345678",
  "event_type": "RISK-001",
  "user_id": "usr_alb2c3d4-e5f6-7890-abcd-ef1234567890",
  "channel": "sms",
  "priority": 1,
  "status": "DELIVERED",
  "template_id": "RISK-001-v3",
  "provider": "msg91",
  "external_id": "msg91_5f6g7h8i9j0k",
  "delivery_attempts": 1,
  "rendered_content": {
    "body": "MARGIN CALL: Shortfall of Rs 1,25,000. Deadline:
11:30 AM. Add funds or positions may be squared off at 12:00
PM. -WealthBridge"
  },
  "state_history": [
```

```
{ "status": "CREATED", "timestamp":  
"2025-03-19T10:15:23.512Z", "actor": "event_ingestion" },  
  { "status": "ENRICHED", "timestamp":  
"2025-03-19T10:15:23.534Z", "actor": "enrichment_worker" },  
  { "status": "ROUTED", "timestamp":  
"2025-03-19T10:15:23.548Z", "actor": "routing_engine",  
"metadata": { "channels_selected": ["sms", "push", "in_app"] }  
},  
  { "status": "QUEUED", "timestamp":  
"2025-03-19T10:15:23.556Z", "actor": "rabbitmq_publisher" },  
  { "status": "SENT", "timestamp":  
"2025-03-19T10:15:24.012Z", "actor": "sms_delivery_worker",  
"metadata": { "provider": "msg91", "external_id":  
"msg91_5f6g7h8i9j0k" } },  
  { "status": "DELIVERED", "timestamp":  
"2025-03-19T10:15:27.891Z", "actor": "dlr_webhook_handler",  
"metadata": { "dlr_status": "DELIVRD" } }  
],  
"compliance": {  
  "dnd_checked": true,  
  "dnd_check_timestamp": "2025-03-19T10:15:23.990Z",  
  "dnd_result": "NOT_REGISTERED",  
  "classification": "TRANSACTIONAL",  
  "regulatory_override": true,  
  "frequency_cap_checked": true,  
  "frequency_cap_result": "WITHIN_LIMITS"  
},  
"cost_paisa": 18,  
"created_at": "2025-03-19T10:15:23.512Z",  
"delivered_at": "2025-03-19T10:15:27.891Z",  
"latency_ms": 4379  
}
```


APPENDIX B: SEQUENCE DIAGRAMS FOR KEY FLOWS

You must create these sequence diagrams in your docs/sequence-diagrams/ directory using Mermaid or Draw.io. The descriptions below define exactly what each diagram must show.

Critical Notification Flow (Margin Call)

This diagram must trace the complete lifecycle of a RISK-001 (Margin Call Warning) event from the trading engine to user delivery:

- Margin Engine → publishes RISK-001 event to Kafka topic 'notification-critical'
- Critical Consumer Group → picks up event within < 100ms (dedicated consumer, not shared)
- Enrichment Service → resolves user context from Redis cache (name, phone, language, timezone, preferences)
- DND Service → checks DND status; RISK-001 is TRANSACTIONAL, bypasses DND even if registered
- Frequency Cap Service → checks caps; CRITICAL events bypass all frequency caps (audit log entry created)
- Routing Engine → regulatory override: RISK-001 MUST use SMS + Push; user preferences cannot disable these
- Template Engine → renders personalised content in user's language (e.g., Hindi for hi locale users)
- RabbitMQ → publishes to priority queue with priority=1 (highest) for each channel
- SMS Delivery Worker → sends via MSG91; if fails, circuit breaker checks → failover to Twilio
- Push Delivery Worker → sends via FCM; stores notification in in-app feed simultaneously
- DLR Webhook → MSG91/Twilio calls back with delivery receipt; state updated to DELIVERED
- Analytics Service → records delivery latency, updates real-time counters

Total end-to-end latency target for this flow: < 10 seconds from event to delivery confirmation.

Non-Critical Notification Flow with Frequency Capping

This diagram must show a MKTX-003 (Market Open) event that gets frequency-capped:

- Market Data Feed → publishes MKTX-003 event to Kafka topic 'notification-events'
- Standard Consumer Group → picks up event (shared consumer group, lower priority)
- Enrichment Service → resolves user context; user has already received 8 notifications today
- Frequency Cap Service → global daily cap is 12; user is at 8; ALLOW but reduce remaining budget
- Frequency Cap Service → per-category hourly cap is 3; user received 3 MKTX in last hour; BLOCK
- State Machine → transition to CAPPED status; record cap reason and scheduled delivery time
- Quiet Hours Check → if within quiet hours, queue for morning digest; otherwise queue for next hour window
- Digest Aggregator → if user has 3+ capped notifications, batch into a single digest push notification

Document how your system handles the race condition where a CRITICAL event arrives while a non-critical event is being frequency-cap evaluated.

Provider Failover Sequence

This diagram must show the circuit breaker and failover mechanism during an SMS provider outage:

- SMS Worker → attempts delivery via MSG91; receives 503 Service Unavailable
- Circuit Breaker → increments failure counter; threshold is 5 failures in 60 seconds
- SMS Worker → retry with exponential backoff (1s, 2s, 4s); MSG91 still returning 503
- Circuit Breaker → failure count reaches 5; transitions from CLOSED to OPEN state
- Alert Service → sends alert to operations channel: 'MSG91 circuit breaker OPEN'
- SMS Worker → all new SMS requests automatically routed to Twilio (secondary provider)
- Circuit Breaker → after 60-second OPEN period, transitions to HALF_OPEN

- SMS Worker → sends single test request to MSG91; if success, transitions to CLOSED
- SMS Worker → if test fails, transitions back to OPEN for another 60-second period
- Idempotency Service → ensures messages sent during failover are not re-sent when primary recovers



ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

APPENDIX C: DOCKER COMPOSE REFERENCE CONFIGURATION

Your docker-compose.yml must include all the services below. This reference configuration shows the minimum required setup. You are expected to add health checks, resource limits, and volume configurations.

```
version: '3.9'
services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: notification_engine
      POSTGRES_USER: notification_user
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    ports: ['5432:5432']
    volumes: ['postgres_data:/var/lib/postgresql/data']
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U notification_user']
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}
    --maxmemory 256mb --maxmemory-policy allkeys-lru
    ports: ['6379:6379']
    healthcheck:
      test: ['CMD', 'redis-cli', '-a', '${REDIS_PASSWORD}',
'ping']
      interval: 10s

  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
```

```
image: confluentinc/cp-kafka:7.5.0
depends_on: [zookeeper]
ports: ['9092:9092']
environment:
  KAFKA_BROKER_ID: 1
  KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
  KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:29092,EXTERNAL://localhost:9092
  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
  KAFKA_NUM_PARTITIONS: 6
  KAFKA_DEFAULT_REPLICATION_FACTOR: 1
healthcheck:
  test: kafka-broker-api-versions --bootstrap-server
localhost:9092
  interval: 30s

rabbitmq:
  image: rabbitmq:3.12-management-alpine
  ports: ['5672:5672', '15672:15672']
  environment:
    RABBITMQ_DEFAULT_USER: ${RABBITMQ_USER}
    RABBITMQ_DEFAULT_PASS: ${RABBITMQ_PASSWORD}
  healthcheck:
    test: rabbitmq-diagnostics -q ping
    interval: 30s

app:
  build:
    context: .
    dockerfile: Dockerfile
    target: production
  depends_on:
    postgres: { condition: service_healthy }
    redis: { condition: service_healthy }
    kafka: { condition: service_healthy }
    rabbitmq: { condition: service_healthy }
  ports: ['3000:3000']
  env_file: .env
```

```
deploy:
  resources:
    limits: { cpus: '1.0', memory: 512M }

volumes:
  postgres_data:
```



ZeTheta

ADD EXPERIENCE TO KNOWLEDGE

APPENDIX D: GLOSSARY OF TERMS

Term	Definition
AMQP	Advanced Message Queuing Protocol. The wire-level protocol used by RabbitMQ for message exchange between producers and consumers.
APNS	Apple Push Notification Service. Apple's cloud service for sending push notifications to iOS and macOS devices.
Backpressure	A flow control mechanism where a downstream consumer signals to an upstream producer to slow down when it cannot keep up with the rate of incoming messages.
BRIN Index	Block Range Index in PostgreSQL. A lightweight index type optimised for naturally ordered data like timestamps. Ideal for time-series notification tables.
Circuit Breaker	A fault tolerance pattern with three states (CLOSED, OPEN, HALF_OPEN) that prevents cascading failures by failing fast when a downstream service is unhealthy.
Consumer Group	A Kafka concept where multiple consumers in the same group share partition assignments, enabling parallel processing of a topic.
CQRS	Command Query Responsibility Segregation. Separating write operations (commands) from read operations (queries) to optimise each independently.
DLQ	Dead Letter Queue. A separate queue where messages that cannot be processed after maximum retry attempts are stored for manual investigation.
DLR	Delivery Receipt. A callback from an SMS or messaging provider confirming that a message was delivered to the recipient's device.
DND	Do Not Disturb. In the TRAI context, refers to the National Do Not Call Registry where users register to block unsolicited commercial communications.
EDA	Event-Driven Architecture. A software design paradigm where the flow of the programme is determined by events rather than sequential procedure calls.
Fan-out	A messaging pattern where a single event is delivered to multiple consumers or channels simultaneously.

FCM	Firestore Cloud Messaging. Google's cross-platform messaging solution for push notifications to Android, iOS, and web applications.
GIN Index	Generalised Inverted Index in PostgreSQL. Optimised for indexing composite values like JSONB columns, enabling fast queries on JSON fields.
Idempotency	A property ensuring that performing the same operation multiple times produces the same result as performing it once. Critical for preventing duplicate notifications.
Jitter	Random variance added to retry delays to prevent thundering herd problems when multiple consumers retry simultaneously after a failure.
NPS	Net Promoter Score. A customer loyalty metric ranging from -100 to +100, calculated from the question 'How likely are you to recommend us?'
Partition	A Kafka concept where a topic is divided into ordered, immutable sequences of records. Partitions enable parallel processing and are the unit of parallelism.
PII	Personally Identifiable Information. Data that can identify an individual, including phone numbers, email addresses, and financial account details.
Saga Pattern	A pattern for managing distributed transactions using a sequence of local transactions with compensating actions for rollback.
SEBI	Securities and Exchange Board of India. The regulatory authority for securities and commodity markets in India.
TRAI	Telecom Regulatory Authority of India. The regulatory body governing telecommunications including SMS and voice call regulations.

ADD EXPERIENCE TO KNOWLEDGE

END OF PROJECT DOCUMENT